

DS 340 Final Project Paper

Author : Seokhoon Shin(U26796848), Joshua Nahm(U91939289)

We contributed equally to this project, collaborating on all aspects including coding, creating the presentation, delivering the presentation, writing the proposal, and drafting the report.

Introduction

The project aimed to address the challenge of selecting appropriate clothing based on prevailing weather conditions, an issue that is often overlooked by traditional weather apps. While such apps are excellent at forecasting weather, they rarely provide actionable advice, such as suggesting what to wear. This gap motivated us to design a system that could classify weather conditions from images and recommend suitable clothing based on the detected weather. By combining deep learning and transfer learning techniques, our project sought to bridge the gap between raw weather data and practical, user-centric recommendations.

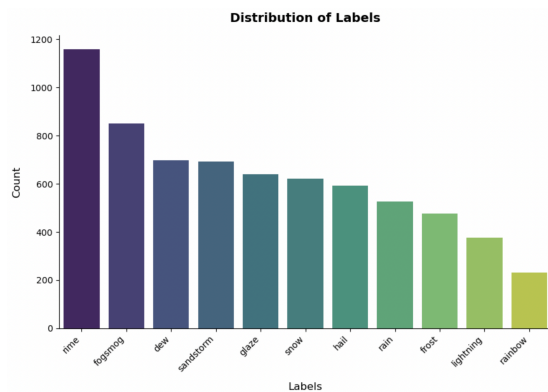


Figure 1: Distribution of weathers

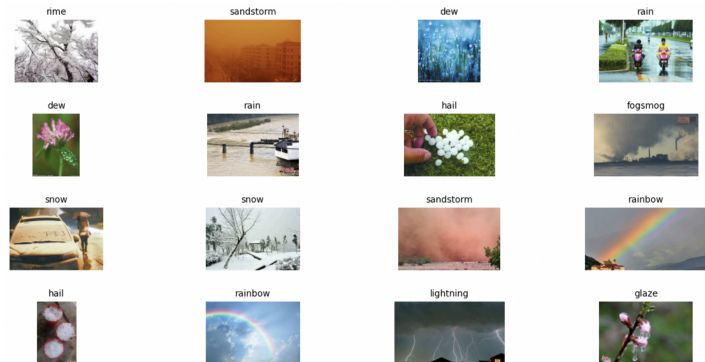


Figure 2: Random images of labels

To analyze the dataset we got from Kaggle and scope of the problem, we examined the distribution of weather labels (**Figure 1**). The dataset comprises 11 unique weather categories, including common types such as rime and fogsmog, as well as less frequent phenomena like rainbows and lightning. This uneven class distribution underscored the importance of using robust models capable of handling imbalanced datasets effectively.

Additionally, we visualized random images from the dataset (**Figure 2**) to explore the diversity and complexity of the weather conditions. The visualizations revealed intricate features such as lighting, textures, and spatial patterns that differentiate one weather type from another, further validating the need for advanced image classification techniques.

The problem necessitated leveraging advanced image classification techniques, as weather conditions often involve nuanced visual elements such as lighting, texture, and spatial patterns. Background research revealed that deep learning, especially convolutional neural networks (CNNs), excels in image-related tasks. Furthermore, transfer learning with pre-trained models such as ResNet50 and VGG19 has been shown to deliver robust results even on limited datasets. This provided the foundation for our approach, which combined a custom CNN as a baseline and state-of-the-art pre-trained models for improved accuracy.

Methodology

The project began with curating and preparing a dataset containing images representing 11 distinct weather conditions, including categories such as rain, snow, sandstorm, and fog. Each image was resized to 100x100 pixels to ensure uniformity in input dimensions for the machine learning models. This preprocessing step was crucial for optimizing computational efficiency and compatibility with the convolutional neural network (CNN) architectures used in the project.

To mitigate overfitting and improve the generalizability of the models, we applied a suite of data augmentation techniques. These included random rotations up to 30 degrees, zooming within a range of $\pm 15\%$, horizontal flipping, and shifting the image width and height by up to 20%. Such augmentations aimed to simulate variations in camera angles, lighting, and environmental factors, effectively enriching the dataset with diverse representations of each weather category.

As an initial step, we developed a custom CNN to establish a baseline performance. This model consisted of three convolutional layers, each followed by max-pooling layers to downsample feature maps, thereby reducing computational complexity. The architecture concluded with dense layers connected to a softmax activation function to perform multi-class classification. While this model achieved no big difference between the training accuracy and test accuracy, its overall low accuracies lead to use of more dense models.

To address this, we employed transfer learning by fine-tuning pre-trained models—VGG19, ResNet50, and MobileNet—originally trained on the ImageNet dataset. These models were chosen for their established ability to extract robust features from image data. We froze the convolutional base layers to retain their pre-trained feature extraction capabilities while adding custom dense layers to adapt the models to our specific classification task. This approach leveraged the general knowledge embedded in the pre-trained weights while allowing the new dense layers to specialize in distinguishing the weather categories.

The models were trained using the Adam optimizer, known for its adaptive learning rate capabilities, and categorical cross-entropy loss, which is well-suited for multi-class classification tasks. To prevent overfitting, we implemented early stopping, halting training when the

validation loss plateaued. Performance evaluation was conducted using accuracy as the primary metric, supplemented by precision, recall, and F1-score to gain insights into the models' ability to handle imbalanced categories. This methodological framework enabled us to iteratively refine our approach and identify the most effective model architecture for the task.

Results

The results of our experiments highlighted the strengths and limitations of different approaches to weather classification using deep learning. Our baseline custom CNN achieved a training accuracy of approximately 64% and a test accuracy of 66%. While this provided a foundation for comparison, the model's simple architecture limited its ability to capture the dataset's complexity, as reflected by its early plateau in validation accuracy. This inability to generalize effectively to unseen data demonstrated the need for more advanced architectures.

Custom CNN (Baseline)

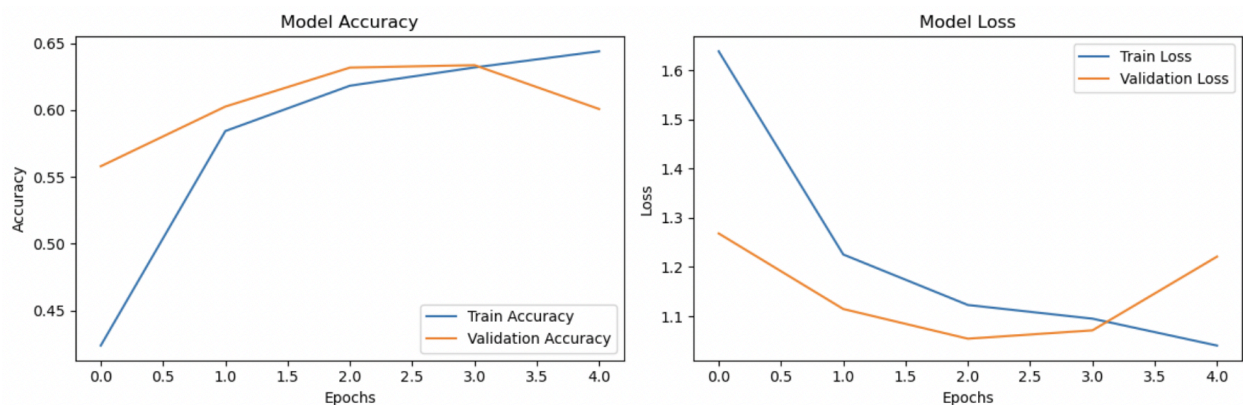


Figure 3: Accuracy & Loss for CNN model

As shown in **Figure 3**, the custom CNN's training accuracy improved gradually over epochs but remained significantly lower than the advanced transfer learning models. Its validation accuracy plateaued early, and the corresponding loss curves further indicate that the model struggled with generalization. These results highlight the custom CNN's limitations in handling nuanced weather conditions, primarily due to its simpler architecture and lack of depth.

Transfer Learning Models

To overcome the shortcomings of the custom CNN, we employed transfer learning with pre-trained models—VGG19, MobileNet, and ResNet50—fine-tuning them for our specific weather classification task. These models demonstrated significant improvements in accuracy and generalization, and their individual performances are summarized below.

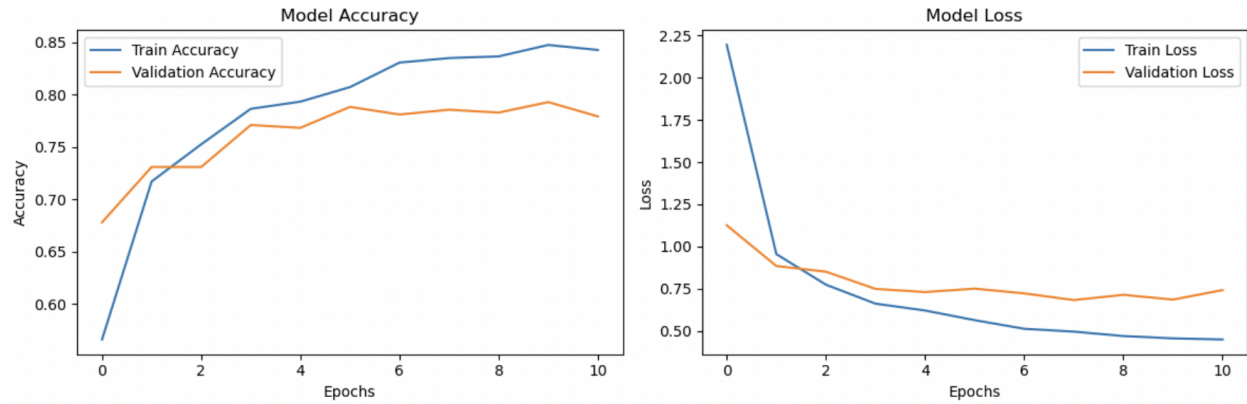


Figure 4: Accuracy & Loss for VGG19 model

VGG19

VGG19 achieved a test accuracy of 77%, showing notable improvement over the custom CNN. As seen in **Figure 4**, VGG19 exhibited better generalization due to its deeper convolutional layers. However, the accuracy curves suggest some degree of underfitting, as training accuracy improved slowly and never fully converged. The validation accuracy improved marginally across epochs, and the loss curves reflect a gradual decrease. While effective, VGG19’s relatively shallow convolutional structure limited its ability to capture intricate weather patterns.

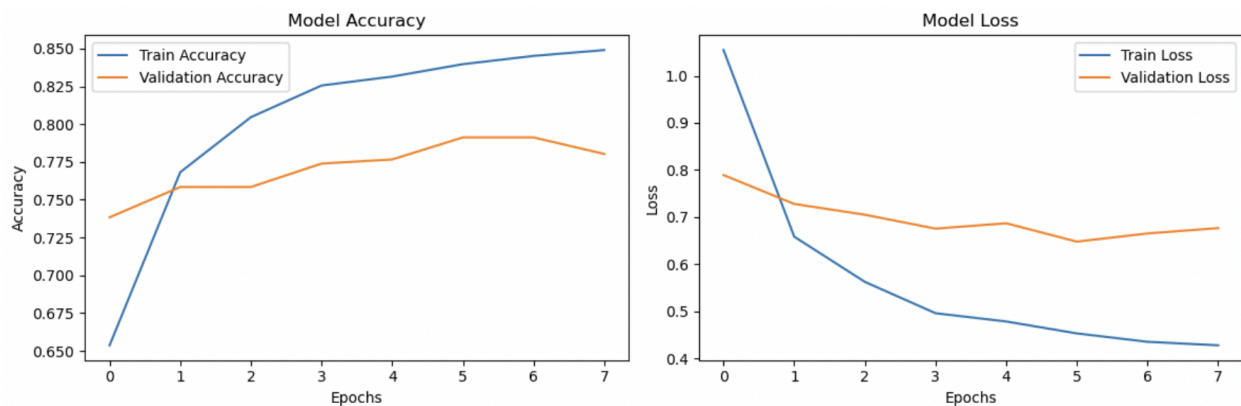


Figure 5: Accuracy & Loss for MobileNet model

MobileNet

MobileNet, chosen for its computational efficiency, achieved a test accuracy of 80%. As illustrated in **Figure 5**, the model’s training accuracy increased rapidly, but a larger gap emerged between training and validation accuracy, suggesting a higher degree of overfitting compared to VGG19. Despite this, MobileNet’s lightweight architecture makes it suitable for deployment in resource-constrained environments. However, its performance revealed a trade-off between

computational efficiency and robustness, as the model occasionally struggled with generalization.

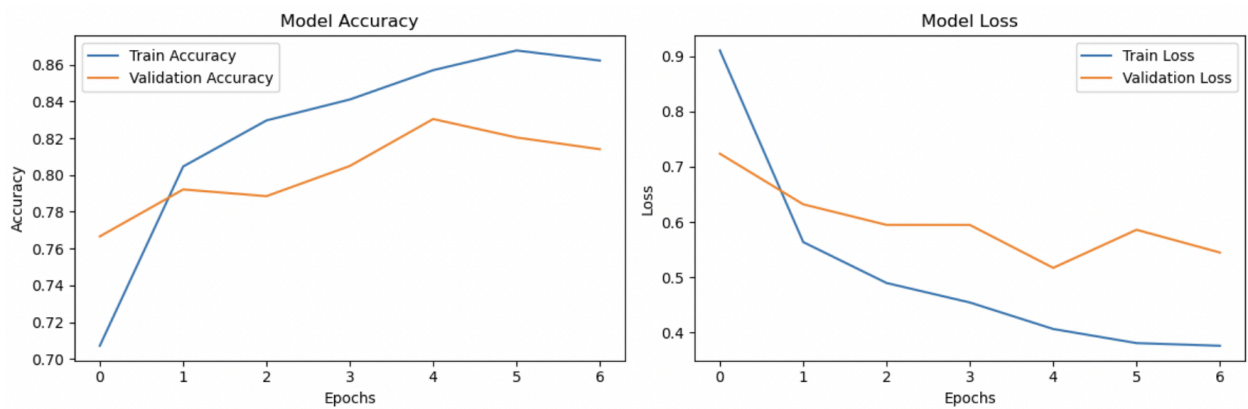


Figure 6: Accuracy & Loss for ResNet50 model

ResNet50

ResNet50 emerged as the best-performing model, achieving a test accuracy of approximately 83%. Its innovative use of skip connections enabled it to mitigate the vanishing gradient problem, allowing the network to extract complex features from the data. As shown in **Figure 6**, the training and validation accuracy curves are closely aligned, indicating excellent generalization. The loss curves further support this observation, with both training and validation loss decreasing consistently across epochs. ResNet50 excelled in identifying distinct weather categories, such as rainbows and sandstorms, though subtle distinctions (e.g., frost vs. dew) occasionally led to misclassifications, as revealed by the confusion matrix (**Figure 7**).

	precision	recall	f1-score	support
dew	0.97	0.87	0.92	133
fogsmog	0.88	0.90	0.89	169
frost	0.59	0.76	0.67	89
glaze	0.75	0.66	0.70	120
hail	0.86	0.92	0.89	125
lightning	0.96	0.98	0.97	82
rain	0.94	0.78	0.85	113
rainbow	0.98	0.89	0.93	47
rime	0.78	0.90	0.84	250
sandstorm	0.85	0.92	0.89	117
snow	0.80	0.58	0.67	128

Figure 7: Confusion Matrix for ResNet50

Application of ResNet50 to External Dataset

Building on ResNet50's success, we applied the model to an external dataset of weather images to assess its generalizability. The model achieved a test accuracy of 83.54%, confirming its robustness across diverse weather categories. Furthermore, we extended the project's scope to recommend appropriate clothing based on the classified weather conditions. For example, as shown in **Figure 8**, the model linked categories such as snow and hail to recommendations for thick clothing, while rain prompted suggestions for umbrellas. This practical application underscores the potential of deep learning to bridge the gap between weather classification and actionable advice, offering a meaningful solution to a real-world problem.

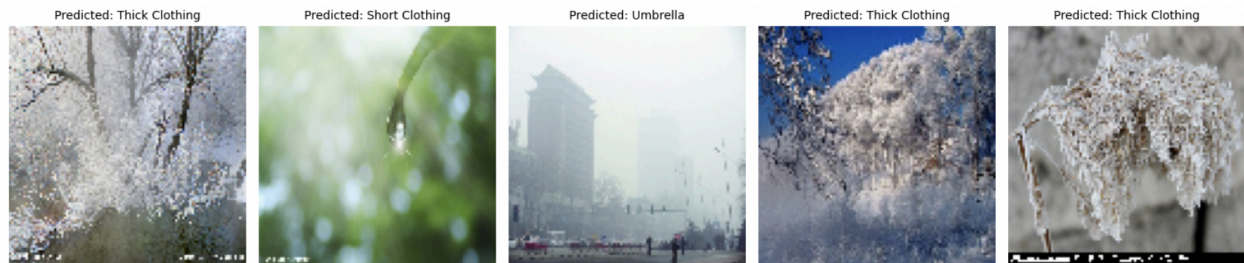


Figure 8: Recommendation for Clothing

Conclusions

Our results demonstrate that leveraging pre-trained models through transfer learning can significantly enhance performance in image classification tasks, especially when data is limited. ResNet50's superior accuracy and minimal overfitting make it a reliable choice for weather classification, outperforming both the custom CNN and other pre-trained architectures. This project reinforced the value of using advanced architectures and data augmentation techniques to improve generalization and handle real-world variability.

Beyond achieving high accuracy, this project provided valuable insights into the importance of balancing model complexity with dataset size. While MobileNet offers computational efficiency, it may require additional regularization to prevent overfitting. Similarly, VGG19, though simpler, showed small signs of overfitting, suggesting that deeper architectures like ResNet50 are better suited for capturing the nuances of weather-related images.

Looking forward, the system could be enhanced by expanding the dataset to include more diverse weather conditions and incorporating real-time prediction capabilities. Exploring newer architectures such as Vision Transformers might further improve performance, particularly for datasets with high variability. By providing actionable advice, such as clothing recommendations, the system bridges the gap between weather data and its practical use, enhancing user convenience and decision-making.

Overall, this project successfully demonstrated the potential of combining image classification with practical recommendations, paving the way for more user-centric applications in machine learning. Through this project, we learned the importance of effectively framing the problem, which allowed us to translate real-world challenges into clear machine learning tasks with specific objectives and well-structured data preparation.

References

- [1] Simonyan, K., & Zisserman, A. (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition." *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1409.1556>
- [2] Howard, A. G., et al. (2017). "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications."
- [3] TensorFlow Documentation. "Transfer Learning and Fine-Tuning with TensorFlow." https://www.tensorflow.org/tutorials/images/transfer_learning
- [4] Keras Documentation. "Applications - Pre-trained Convolutional Neural Networks in Keras." <https://keras.io/api/applications/>
- [5] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org/>
- [6] Geron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd Edition. O'Reilly Media.
- [7] Albumentations Documentation. "Data Augmentation Techniques for Image Classification." <https://albumentations.ai/docs/>
- [8] OpenAI. "ChatGPT – AI Language Model for Grammar Assistance and Code Optimization."
- [9] Bhathena, Jehan. "Weather Dataset." Kaggle. <https://www.kaggle.com/datasets/jehanbhathena/weather-dataset/data>
- [10] Scikit-Learn Documentation. "Classification Metrics." https://scikit-learn.org/stable/modules/model_evaluation.html